BET Cheatsheet

Construct an estimator

```
from BayesianEstimationTool import Estimator, Nuts, Svi, ValidateMethod
est = Estimator(data, ModelFn, localZNames=["growth"],
                GlobalPriorFn=GlobalPriorFn, rngSeed=1)
```

All arrays in `data` share their first axis (individual). `ModelFn(globals, localZs, oneIndividualData)` returns a dictionary. `GlobalPriorFn(dataShapes)` returns a dictionary of NumPyro distributions. Likelihood functions return `{site: (observation, distribution)}`.

Train and test

```
method = Nuts(numWarmup=2000, numSamples=2000, numChains=4)
# Or: Svi(numSteps=10000, numSamples=2000)
ValidateMethod(method)
est.Train(trainIdxs, TrainLikelihoodFn, method, saveLocalZs=True)
est.Test(testIdxs, TestLikelihoodFn, method, nGlobalSamples=None)
# Without observations: est.Test(testIdxs, nLocalZSamples=2000)
```

`nGlobalSamples=None` uses global posterior medians. An integer selects stored global posterior draws. `nLocalZSamples` controls local prior draws in the no-likelihood test mode.

Scores and shapes

```
train = est.ScoreTrainSamples(ScoreFn, nSamples=50, rngSeed=3)
test = est.ScoreTestSamples(ScoreFn, nSamples=50, rngSeed=3)
```

Training: `[posterior_draw, individual, ...score_shape]`. Testing: `[global_draw, local_draw, individual, ...score_shape]`. `ScoreFn(modelOut, oneIndividualData)` returns scalar or array.

Save and load

```
est.Save("fit.npz", saveAllData=True)
restored = Estimator.Load("fit.npz", ModelFn=ModelFn,
                          GlobalPriorFn=GlobalPriorFn)
```

Model and prior functions must be supplied at load time. If `saveAllData=False`, pass `data=` on load.

Mask padded or hidden observations

```
valid = data["valid"]  # one individual's boolean observation mask
pred = modelOut["prediction"]
safe_y = jnp.where(valid, data["y"], pred)
likelihood = dist.Normal(pred, sigma).mask(valid)
return {"y": (safe_y, likelihood)}
```

Store variable-length sequences as padded arrays `[individual, time]`. Hide additional observations in `TestLikelihoodFn` with a stricter mask. Use finite safe placeholders and finite model predictions; masking does not make NaN intermediate calculations safe.

JAX and ODE reminders

- Model functions must be JAX-traceable; prefer `jax.numpy`, `vmap`, and `jax.lax` over Python operations on tracers.
- For ODEs, use JAX-compatible `jax.experimental.ode.odeint` or optional Diffrax; check sorted time grids and differentiability.

- Inspect parameter transforms, array axes, observation masks, noise scales, and nonfinite gradients before interpreting inference results.
- Local Z variables are standardized normal; with multiple variables BET estimates an LKJ Cholesky correlation structure. Transform local Zs into physical parameters inside `ModelFn`.

Test suite

From the parent of the repository: `JAX_PLATFORMS=cpu python -m pytest -q BayesianEstimationTool/tests`. See `Manual.md`, `METHOD_AUTHORING.md`, and `TESTING.md` for details.